
boolean

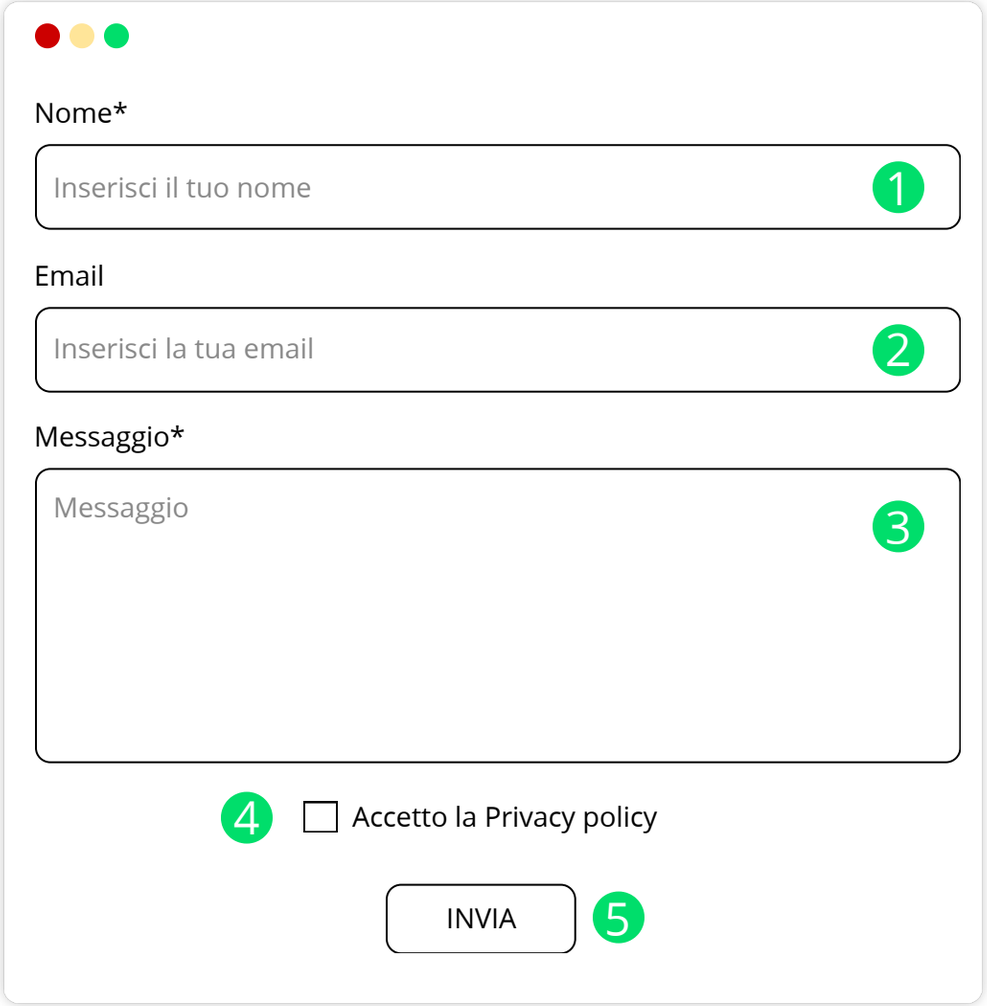


FORM

Form

Ricevere dati dall'utente

1. Input Text
2. Input Email
3. Textarea
4. Checkbox
5. Button



Form

Ricevere dati dall'utente

Nome*

Inserisci il tuo nome

Email

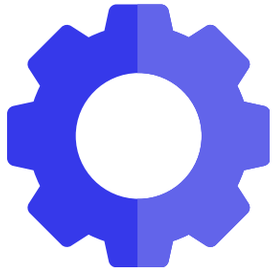
Inserisci la tua email

Messaggio*

Messaggio

Accetto la Privacy policy

INVIA



Al click del tasto INVIA, tutto il contenuto del form viene inviato ad una parte del backend che svolgerà le opportune azioni.

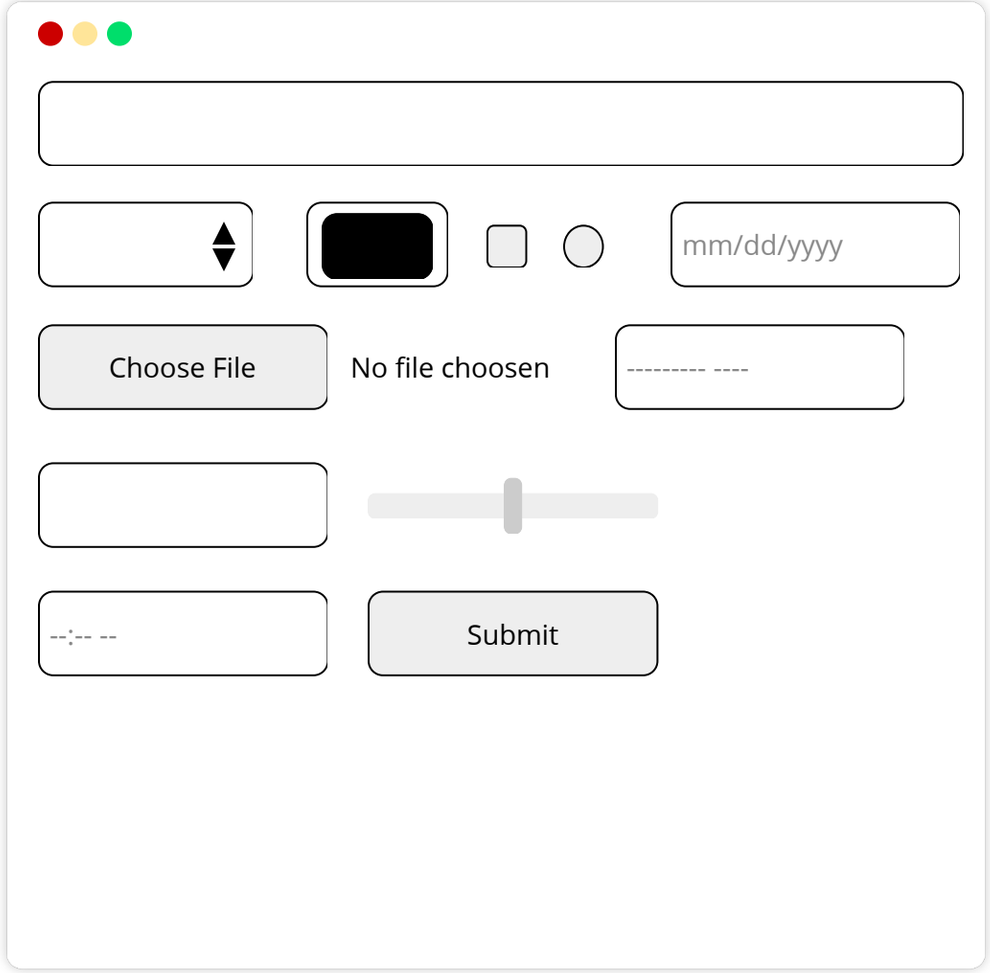
In questo caso, salvare le informazioni sul database e inviare una email.

Form

I tag

```
<!-- <input type="tipo"> -->

<input type="text">
<input type="number">
<input type="color">
<input type="checkbox">
<input type="radio">
<input type="date">
<input type="file">
<input type="month">
<input type="password">
<input type="range">
<input type="time">
<input type="submit">
```

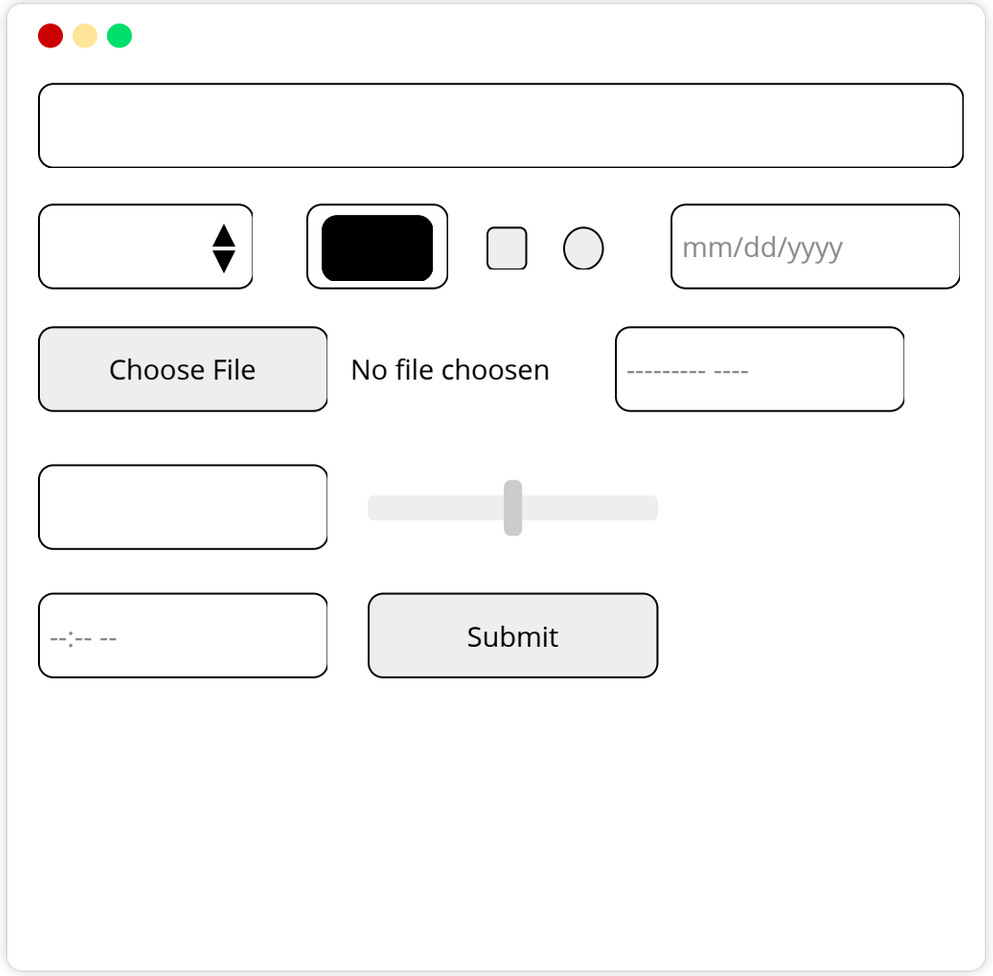


Form

I tag

```
<!-- <input type="tipo"> -->

<input type="text">
<input type="number">
<input type="color">
<input type="checkbox">
<input type="radio">
<input type="date">
<input type="file">
<input type="month">
<input type="password">
<input type="range">
<input type="time">
<input type="submit">
```

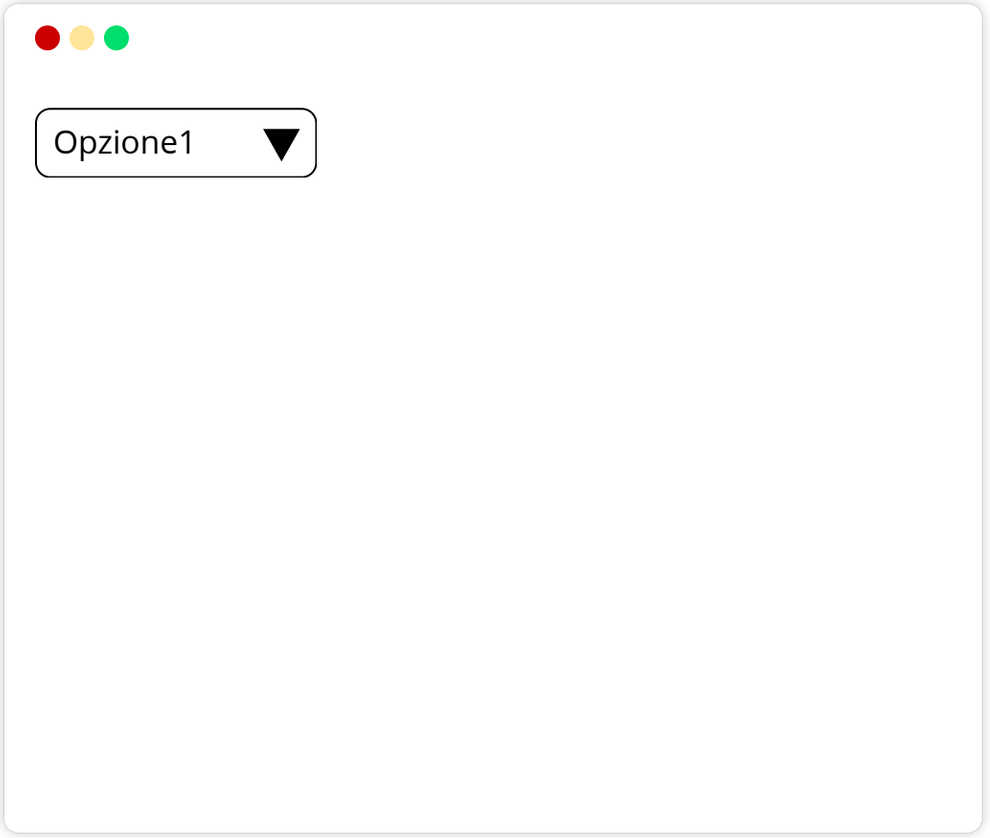


Tipi di Input

Select

```html
<select name="scelta">
  <option value="opt1">Opzione1</option>
  <option value="opt2">Opzione2</option>
</select>

```



Tipi di Input

Textarea

```
1 <textarea name="name" rows="8" cols="80">
2 </textarea>
3
```

Possiamo inserire un testo più lungo disposto su
più righe

Form Validation

HTML5 identifica alcune parole chiave per validare il contenuto dei form



Di default, i browser controllano che ci sia corrispondenza tra il "type" dichiarato e ciò che viene inserto, required obbliga la presenza di un valore, minlength, maxlength forzano un numero minimo e un numero massimo di caratteri (o ancora meglio l'attributo "pattern").

In caso il contenuto non sia valido, il form non viene inviato.

Queste regole sono specifiche di HTML5. Non funzionano su browser più datati e sono più facilmente aggirabili, quindi alla client-side validation va sempre affiancata la server-side validation (es. live)



LIVE CODING



Form e JavaScript

Aka: Come lavorare coi campi dei form

Form e JS

Leggere il contenuto dei campi di un form

Tutti i tag tipici dei form (come `<input>`, `<select>`, `<textarea>`, ecc.) ci danno la possibilità di leggere il valore al loro interno tramite l'attributo **`value`**.




```html
<input id="name" value="Marco">
```

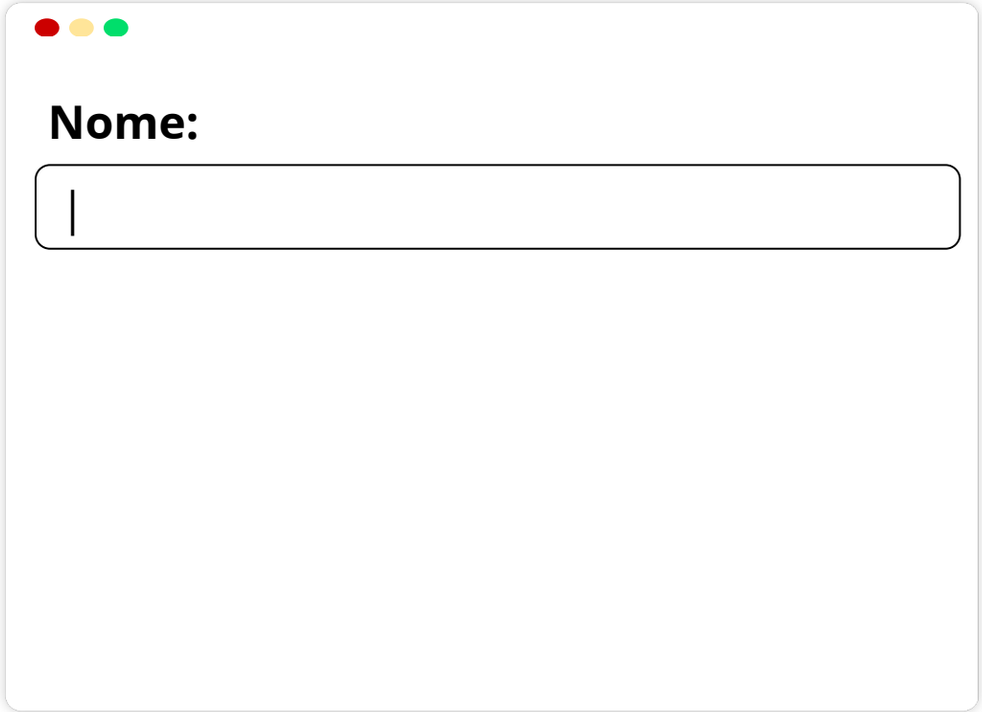
```js
const field = document.getElementById('name');

const name = field.value; // Marco
```

Form e JS

Leggere il contenuto dei campi di un form

Il problema è che all'avvio della pagina, i nostri campi sono quasi sempre vuoti, perchè l'utente deve ancora scriverci dentro! Di conseguenza, **eseguire questo codice darebbe sempre stringa vuota!**




```html
<input id="name">
```

```js
const field = document.getElementById('name');

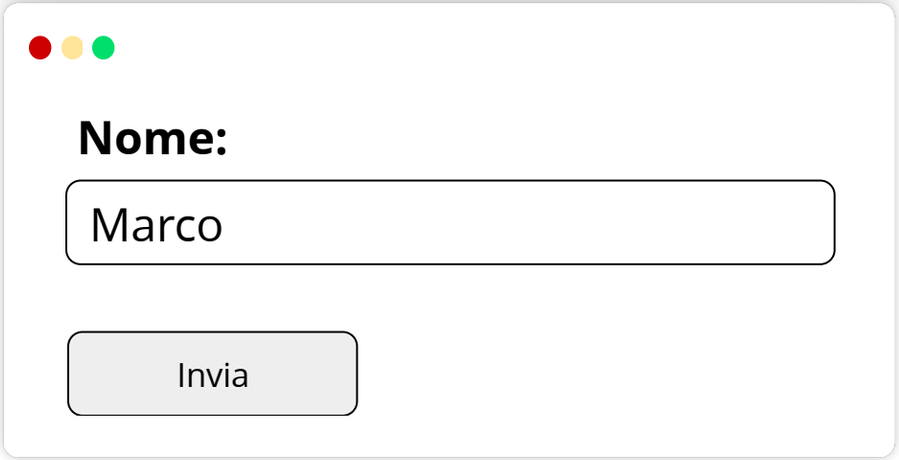
const name = field.value; // ''
```

Form e JS

Leggere il contenuto dei campi di un form

Fortunatamente, a noi interessa leggere il contenuto solo dopo che l'utente invia il form, pertanto possiamo sfruttare un button e un **eventListener**!

```html
<input id="name" value="Marco">
<button>Invia</button>
```




```js
const field = document.getElementById('name');
const button = document.querySelector('button');

// se proviamo a leggere qui, sarà sempre vuoto
const name = field.value; //

button.addEventListener('click', () => {
  const name = field.value; // Marco
});
```



LIVE CODING

Form

E il <form>?

Tutti gli input che vogliono essere inviati devono essere incapsulati in un tag `<form>`

Solo gli input che sono all'interno vengono inviati all'indirizzo definito in `action.`

Se il valore di `action` è vuoto, la pagina invia i dati a se stessa e si ricarica.

Anche il button deve essere all'interno del form affinchè ne invii i campi quando lo clicchiamo.

```
1 <form action="www.miosito.com/boh...">
2
3   <input type="text">
4   <button>Send Me</button>
5
6 </form>
7
```

Form

Il Tag <form> e il suo comportamento naturale

Quando il browser capisce che stiamo inviando un form, a prescindere dal valore di action, effettua una nuova richiesta e quindi **perdiamo tutti i dati JavaScript**, come se avessimo aggiornato la pagina.

Questo succede perchè normalmente **i form sono nati per inviare dati da una pagina a un'altra**. E spesso l'altra pagina è una pagina di backend.

Tuttavia, **noi vogliamo gestire i dati con JavaScript**, restando nella nostra pagina.. come possiamo fare?

Form

Giù le mani dal mio form!

La soluzione consiste nell'intercettare l'evento di invio del button (evento `'submit'`) e impedire che effettui il suo comportamento di default!

Sfruttiamo un event listener all'evento `submit` del form.

Sfruttiamo il parametro `event`, che ci viene fornito "magicamente" da JS.

Blocchiamo il comportamento naturale con `event.preventDefault()`

```
1  // Recupero il form
2  const form = document.querySelector('form');
3
4  // Intercetto l'evento di invio
5  form.addEventListener('submit', (event) => {
6    // Blocco l'invio del form..
7    event.preventDefault();
8
9    // Tutto il resto del codice...
10 });
```



LIVE CODING

ESERCITAZIONE